



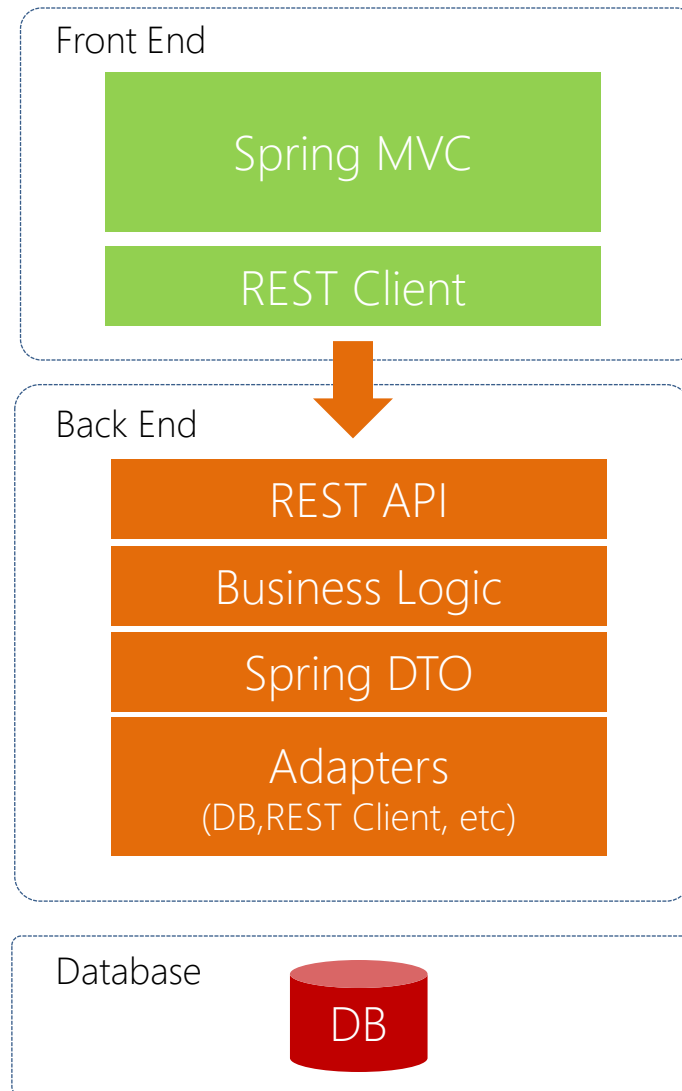
# Online Account Opening

## Collaboration Development

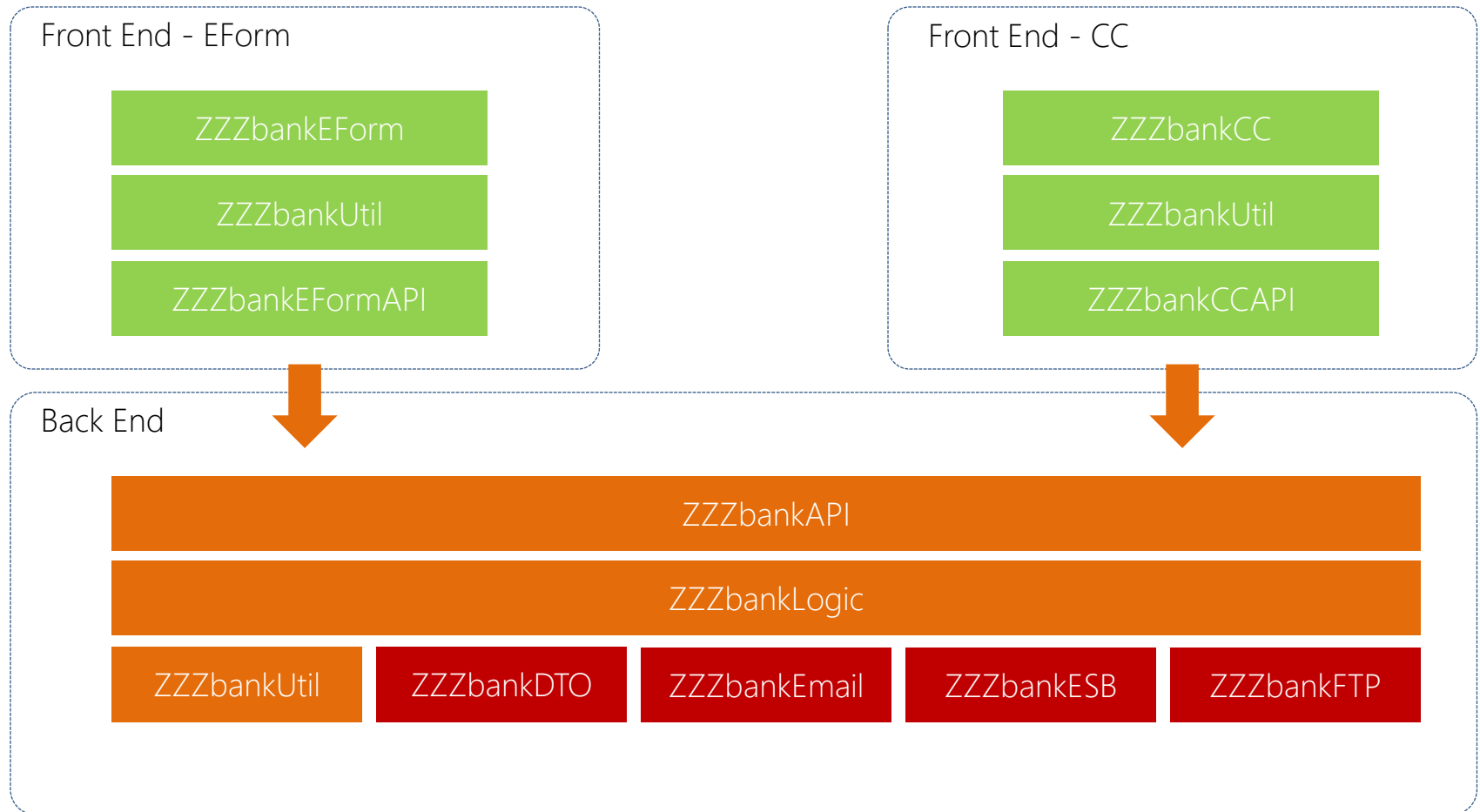
Online Account Opening

# DEVELOPMENT CONVENTION

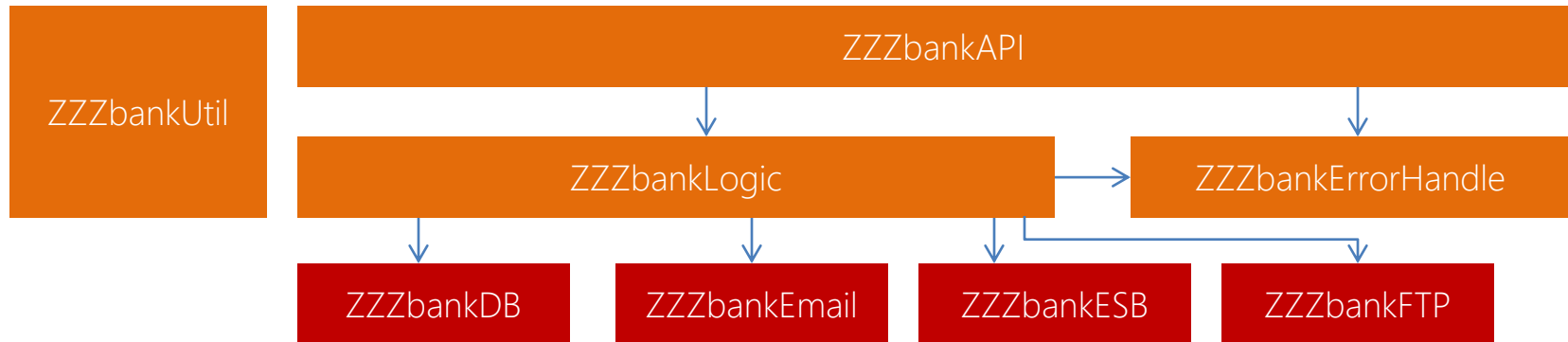
# High Level Architecture Design



# Module Architecture Design

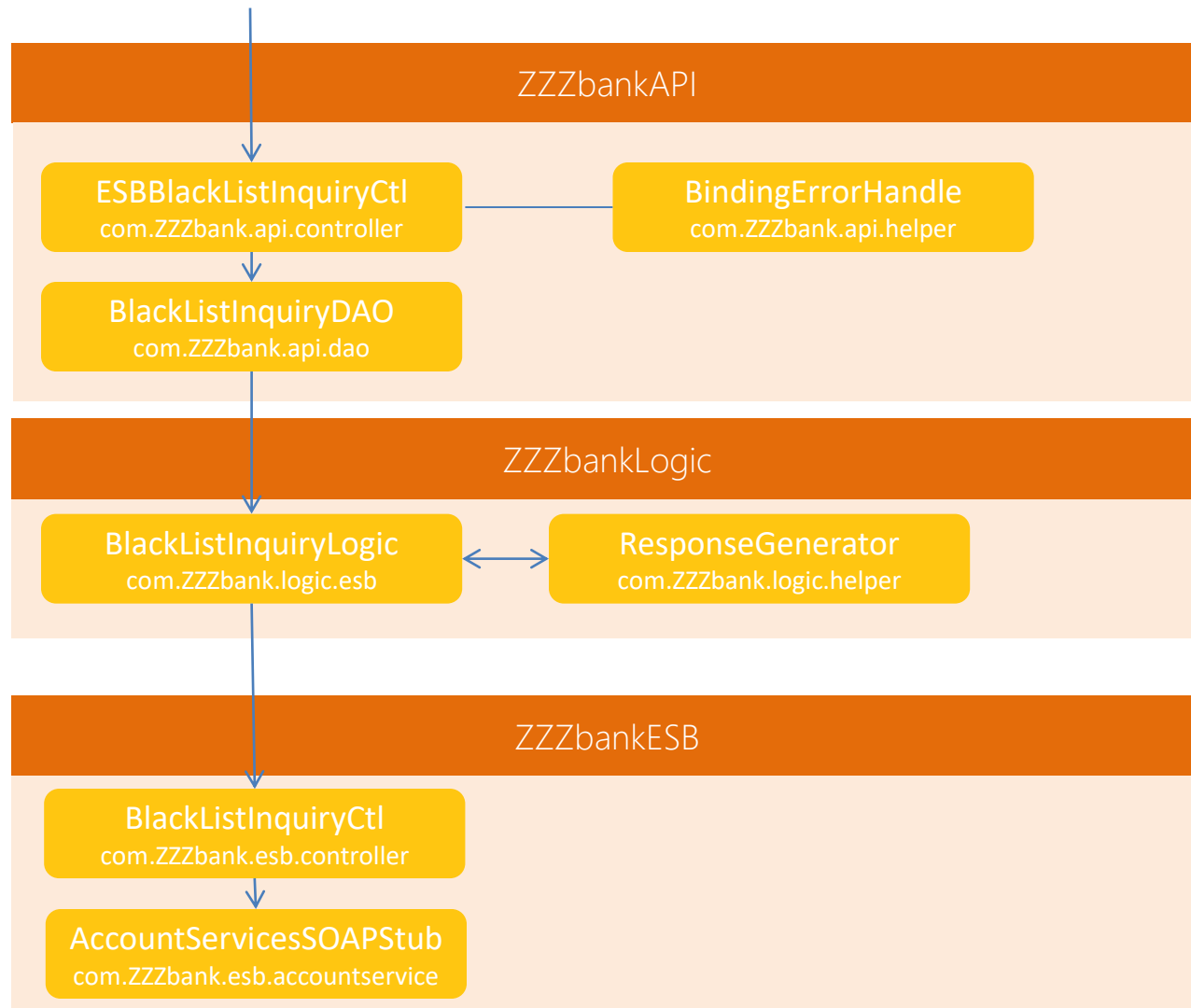


# STP Backend Architecture



- ZZZbankUtil is used by any other component
- External System Access STP Backend thru ZZZbankAPI
- ZZZbankAPI accessing adapter through ZZZbankLogic
- All Error Message shall be transformed to proper Response

Message by ZZZbankErrorHandle



# Error Handle Sample

ZZZbankLogic

```
// found error  
respHeader =  
respMessageController.getResponseHeader(null,"java.net.timeOutException","ESB")'
```

ZZZbankErrorHandle

com.ZZZbank.errorhandle.com.oller.ResponseMessageController

```
public Hashtable getResponseHeader(String code,String message, String module){  
    Hashtable responseHeader = new Hashtable();  
    if (code == null) || (code.equalsIgnoreCase("")){  
        // get response by code  
        String respDesc = getRespMessageByCode (module,code);  
    } else {  
        // get response by message  
        String respDesc = getRespMessageByMessage (module,message);  
    }  
    return responseHeader;  
}
```

# Architecture Consideration

- 3 tier Application, segregation between
  - Front-End
  - Back-End
  - and Data Layer (DB)
- Security Compliance
- Modularity
  - Code Classification
  - Ease On Future Development
  - Maintainability



# Code Convention

- Following Java Convention Name, sample :
  - Project : refer to naming from SUN → ZZZbankDTO
  - Package : com.ZZZbank.eform
  - Class : BaseController
  - Method : getAccountInformation
  - Variable : accountNo
  - Constants : ACCOUNT\_NO

# Java Project Definition

- **ZZZbankDTO** : DTO Classes
- **ZZZbankLogic** : Application logic classes which call DTO or other Service Adapters to external system
- **ZZZbankEForm** : E-Form Classes and Web Artifact (Spring MVC)
- **ZZZbankCC** : Customer Care Classes and Web Artifact (Spring MVC)
- **ZZZbankAPI**: Backend Server API, conduct direct call to Logic Classes
- **ZZZbankESB**: ESB Integration Adapter Classes
- **ZZZbankEmail**: Email Integration Classes
- **ZZZbankFTP**: FTP Integration Classes
- **ZZZbankUtil**: Multi Purpose Utility Classes

# RESTful Web Service Response

@HTTPHeader

HTTP 200 OK

@ResponseBody

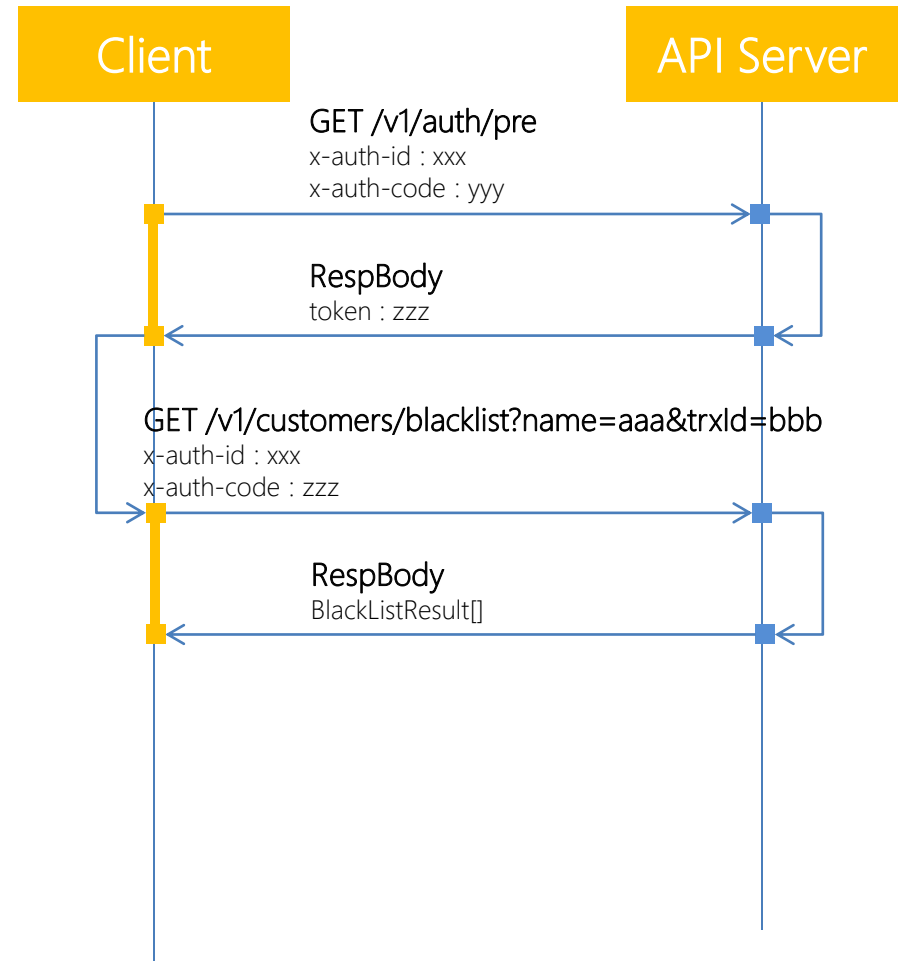
```
{  
  "Response":  
  {  
    "RespHeader":  
    {  
      "respCode": "1",  
      "respDesc": "success"  
    },  
    "RespBody":  
    {  
      "highRiskFlag": "true"  
    }  
  }  
}
```

Online Account Opening

**SECURED INTERFACE**

# REST API Security

- Leverage JWT (JSON Web Token) as security solution
- Benefits : does not store on Server, lesser space on Storage and Memory
- How does it works :
- **API Call 1 :**
  - Client call Pre-Authenticate API, provide secret key, combined with client datetime
  - API Server validate the secret key and provide JWT String should it be valid
- **API Call 2 :**
  - Client utilize JWT as one of header param for subsequent request
  - API Server validate the JWT Session
  - If JWT String Valid, Server shall call the "integration interface" and set respCode = 1
- Any failure in validation shall be responded with proper HTTP Status Code (e.g : 400 – Bad Request, 401 – Unauthorized)



# Pre-Authenticate Code

com.ZZZbank.api.controller.AuthSecurityCtl

```
@RequestMapping(value="/auth/pre",produces="application/json",method=RequestMethod.GET)
@ResponseBody
private ResponseEntity getJwtCode(@RequestHeader Map<String,String> headers){
    ...
    Hashtable htResult = authDAO.doValidatePreAuth(headers);
    ...
    responseEntity = new ResponseEntity(htResult,HttpStatusCode.getHttpResonseCode(respCode));
    ...
}
```

REST WS

com.ZZZbank.api.dao.AuthSecurityDAO

```
public Hashtable doValidatePreAuth(Map<String,String> mapHeader){
    return AuthHandler.doPreAuth(mapHeader);
}
```

DAO

com.ZZZbank.api.security.AuthHandler

```
public static Hashtable doPreAuth(Map<String,String> mapHeader){
    ...
    if ((strCmp==strSecretKey) || (strCmp.equals(strSecretKey))){
        String strJwtCode = JwtHandler.doGenerateJwt(strSecretKey, systemId, userId);
    }
    ...
}
```

Security

Online Account Opening

# SAMPLE ESB INTEGRATION

# Pre-Auth Sample : Positive Case

The screenshot shows the Postman interface for an API call to `http://localhost:8081/0.1/auth/pre`. The **Headers** tab is active, showing two headers: `x-auth-code` with value `MDAwMDAwMHdlYm1heWJhbmtzdHAyMDE2` and `x-auth-id` with value `web usr`. The **Body** tab is also active, showing a JSON response with a `token` field. The status is **200 OK** and the time is **1499 ms**.

**Header containing secret key and app id**

**Successful Result, Provide Token**

```

{
  "Response": {
    "RespHeader": {
      "respCode": "1",
      "respDesc": "success"
    },
    "RespBody": {
      "token": "eyJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJtYXkiYm1heWJhbmtzdHAyMDE2LjZNSWVhbnVf-68o3LUjAOTLld2ISVY1XpHT1XUfPHFDvmw"
    }
  }
}

```



# Pre-Auth Sample : Negative Case

The screenshot displays a REST client interface with the following components:

- Request Bar:** Method `GET`, URL `http://localhost:8081/.../0.1/auth/pre`, and buttons for `Send` and `Save`.
- Headers Tab:** A table with columns `key` and `value` is shown, but it is empty. A red box highlights this area, with a callout stating "No Header Provided".
- Response Section:**
  - Status Bar:** Shows `Status: 400 Bad Request` and `Time: 171 ms`. A red box highlights the status, with a callout stating "Unsuccessfull Result, HTTP 400, JSON Response".
  - Response Body:** The response is in JSON format, showing an unsuccessful result:

```
1 {
2   "Response": {
3     "RespHeader": {
4       "respCode": "11000",
5       "respDesc": "Missing Pre-Auth Identifier"
6     },
7     "RespBody": {}
8   }
9 }
```

A red box highlights the `respCode` and `respDesc` fields.

# BlackListInquiry (assumed Token Provided)

- Request :

<http://localhost:8081/ZZZbankAPI-0.1/customers/blacklist?name=michael&trxlD=001>

- Response :

```
{ "Response": { "RespHeader": { "respCode": "1", "respDesc": "success", "RespBody": { "blackListCustomers": [ { "id": "201600001", "currentRecordVersion": null, "taxNo": null, "name": "MICHAEL JACKSON", "birthDate": "1972-08-30", "sourceCode": null, "remark": null, "registrationNo": "2003012300001", "addrLine1": null, "addrLine2": null, "addrLine3": null, "blacklistCode": null, "blacklistNo": null, "letterNo": null, "letterDate": null, "createdBy": null, "createdDate": null, "updatedBy": null, "updatedDate": null, "maintenanceBranchCode": null, "operatorId": null, "telephoneNo": null, "historyLetterNo": null, "historyDateLetter": null, "historyName": null, "historyAddrLine1": null, "historyAddrLine2": null, "historyAddrLine3": null, "mtRsnCd": null, "mtBlacklistStatus": null, "motherMaidenName": null, "acctNo": null, "status": null, "clearanceDate": null, "asAtDate": null, "downloadDate": null, "hostSourceMappingList": null, "sourceDescription": null, "expiryDate": null, "nationalityCode": null, "nationalityDescription": null, "companyName": null, "customerName": null, "cifNumber": null, "systemFlag": null, "terroristBirthDate": null, "blacklistFlag": null, "mothersMaidenNameCheck": null, "birthDateCheck": null, "regNoCheck": null }, { "id": null, "currentRecordVersion": null, "taxNo": null, "name": "MICHAEL LEARNS TO ROCK", "birthDate": "1975-12-31", "sourceCode": null, "remark": null, "registrationNo": "20000201678", "addrLine1": null, "addrLine2": null, "addrLine3": null, "blacklistCode": null, "blacklistNo": null, "letterNo": null, "letterDate": null, "createdBy": null, "createdDate": null, "updatedBy": null, "updatedDate": null, "maintenanceBranchCode": null, "operatorId": null, "telephoneNo": null, "historyLetterNo": null, "historyDateLetter": null, "historyName": null, "historyAddrLine1": null, "historyAddrLine2": null, "historyAddrLine3": null, "mtRsnCd": null, "mtBlacklistStatus": null, "motherMaidenName": null, "acctNo": null, "status": null, "clearanceDate": null, "asAtDate": null, "downloadDate": null, "hostSourceMappingList": null, "sourceDescription": null, "expiryDate": null, "nationalityCode": null, "nationalityDescription": null, "companyName": null, "customerName": null, "cifNumber": null, "systemFlag": null, "terroristBirthDate": null, "blacklistFlag": null, "mothersMaidenNameCheck": null, "birthDateCheck": null, "regNoCheck": null }, { "id": null, "currentRecordVersion": null, "taxNo": null, "name": "MICHAEL JORDAN", "birthDate": "1968-01-23", "sourceCode": null, "remark": null, "registrationNo": "19980100001", "addrLine1": null, "addrLine2": null, "addrLine3": null, "blacklistCode": null, "blacklistNo": null, "letterNo": null, "letterDate": null, "createdBy": null, "createdDate": null, "updatedBy": null, "updatedDate": null, "maintenanceBranchCode": null, "operatorId": null, "telephoneNo": null, "historyLetterNo": null, "historyDateLetter": null, "historyName": null, "historyAddrLine1": null, "historyAddrLine2": null, "historyAddrLine3": null, "mtRsnCd": null, "mtBlacklistStatus": null, "motherMaidenName": null, "acctNo": null, "status": null, "clearanceDate": null, "asAtDate": null, "downloadDate": null, "hostSourceMappingList": null, "sourceDescription": null, "expiryDate": null, "nationalityCode": null, "nationalityDescription": null, "companyName": null, "customerName": null, "cifNumber": null, "systemFlag": null, "terroristBirthDate": null, "blacklistFlag": null, "mothersMaidenNameCheck": null, "birthDateCheck": null, "regNoCheck": null } ], "totalPages": 1 } }
```

# CustomerHighRiskInquiry (assumed Token Provided)

- Request :

<http://localhost:8081/ZZZbankAPI-0.1/customers/highrisk?name=michaels&trxID=001&occupationCode=AD&ecoSectorCode=AD>

- Response :

```
{"Response":{"RespHeader":{"respCode":"1","respDesc":"success"},"RespBody":{"highRiskFlag":"true"}}
```